Write Python code for a DNN solving the XOR problem using four input-output pairs

To develop a Python-based Deep Neural Network that learns the XOR function using the four specified input-output pairs.

Algorithm
1. Import NumPy and TensorFlow libraries.
2. Define the four XOR input combinations.
3. Assign the corresponding output values 0, 1, 1 and 0.
4. Convert the input and output values into floating-point arrays.
5. Create a Sequential neural network model.
6. Add a hidden layer containing eight neurons with tanh activation.
7. Add an output layer containing one neuron with sigmoid activation.
8. Compile the model using the Adam optimizer.
9. Use binary cross-entropy as the loss function.
10. Train the model using the four XOR input-output pairs.
11. Evaluate the trained model using the same dataset.
12. Display the predicted output and classification accuracy.

Program
```
import numpy as np
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
tf.keras.utils.set_random_seed(21)
# Input dataset
X = np.array([
[0, 0],
[0, 1],
[1, 0],
[1, 1]
], dtype="float32")
# Output dataset
y = np.array([0, 1, 1, 0], dtype="float32")
# Create neural network
model = Sequential([
Dense(8, input_shape=(2,), activation='tanh'),
Dense(1, activation='sigmoid')
])
# Compile model
model.compile(
optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy']
)
# Train model
model.fit(X, y, epochs=2000, verbose=0)
# Evaluate model
loss, accuracy = model.evaluate(X, y, verbose=0)
# Predict output
prob = model.predict(X, verbose=0).ravel()
pred = (prob >= 0.5).astype(int)
print("XOR DNN Results")
print("Accuracy:", accuracy * 100, "%")
for i in range(len(X)):
print(
X[i].astype(int),

"Actual:", int(y[i]),
"Predicted:", pred[i]
)
```

Output of Program
```
XOR DNN Results
Accuracy: 100.0 %
[0 0] Actual: 0 Predicted: 0
[0 1] Actual: 1 Predicted: 1
[1 0] Actual: 1 Predicted: 1
[1 1] Actual: 0 Predicted: 0
```

Result

Thus, a DNN model was successfully developed and trained using the four XOR input-output pairs. The model learned the XOR classification pattern and produced the expected outputs.